

Figure 3 Panels-ndsl-nupid-pras copy

September 14, 2026

```
[1]: import scanpy as sc
import anndata as ad
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import os
#import scselpy as scS
```

0.0.1 Import schwann with new_mds

```
[3]: schwann = sc.read_h5ad('schwann_new_mds.h5ad')
```

```
[4]: colors = [
    "#3498db", # Royal Blue
    "#e74c3c", # Alizarin Crimson
    "#2ecc71", # Emerald
    "#f39c12", # Sunflower
    "#1abc9c", # Turquoise
    "#9b59b6", # Amethyst
    "#e67e22", # Carrot
    "#27ae60", # Nephritis
    "#c0392b", # Pomegranate
    "#34495e" # Wet Asphalt
]

# ct_dict = {}
# for i in range(0, len(cell_types)):
#     ct_dict[cell_types[i]] = colors[i]

# schwann_dict = {}
# for i in range(0, len(schwann_subclasses)):
#     schwann_dict[schwann_subclasses[i]] = colors[i]

# macro_dict = {}
# for i in range(0, len(macrophage_subclasses)):
#     macrophage_dict[macrophage_subclasses[i]] = colors[i]
```

```

# NF2_spor_dict = {'NF2':colors[3], 'Sporadic':colors[0]}

# Hard code colors so it looks better
ct_dict = {
    'Macrophage':"#3498db",
    'Endothelial':"#e74c3c",
    'T-cells':"#2ecc71",
    'Fibroblast':"#f39c12",
    'Unknown':"#1abc9c",
    'Schwann':"#9b59b6",
    'Pericytes':"#e67e22",
    'Neutrophil':"#27ae60",
    'NK Cells':"#c0392b"
}

schwann_dict = {
    'Macrophage':"#3498db",
    'Endothelial-like':"#e74c3c",
    'IFN_Response':"#2ecc71",
    'Hypoxia':"#f39c12",
    'Repair-like':"#1abc9c",
    'Classical':"#9b59b6",
    'Proliferating':"#27ae60",
    'Myelinating':"#c0392b",
    'Stress': "#34495e"
}

schwann_dict1 = {
    'MHCII':"#3498db",
    'Stress':"#e74c3c",
    'IFN Response':"#2ecc71",
    'Hypoxia':"#f39c12",
    'Repair-like':"#1abc9c",
    'Core':"#9b59b6",
    'Unknown':"#808080",
    'Myelinating':"#c0392b"
}

macro_dict = {
    'M1-like':"#3498db",
    'M2-like':"#e74c3c",
    'Pro-Inflammatory':"#f39c12",
    'Regulatory':"#9b59b6",
    'Proliferating':"#27ae60"
}

macro_dict1 = {
    'Macrophage_M1-like':"#3498db",

```

```

    'Macrophage_M2-like':"#e74c3c",
    'Macrophage_Pro-Inflammatory':"#f39c12",
    'Macrophage_Regulatory':"#9b59b6",
    'Macrophage_Proliferating':"#27ae60"
}
NF2_spor_dict = {'NF2':"#e74c3c", 'Sporadic':"#3498db"}

# Samples per genotype:
#NF2 = scVS01, scVS02, snVS01, snVS02, snVS03, snVS04, snVS05, snVS06, snVS07,
↳snVS08, snVS09
#Sporadic = scVS10, scVS11, snVS10, snVS11, snVS12, snVS13, snVS14, snVS15

# Define color palettes for NF2 and Sporadic
nf2_colors = ['#e74c3c', '#d35400', '#c0392b', '#e67e22', '#f39c12',
              '#cd6155', '#a93226', '#ba4a00', '#f1c40f', '#d68910', '#b9770e']
↳ # 11 colors for NF2

sporadic_colors = ['#3498db', '#2980b9', '#1abc9c', '#2ecc71', '#16a085',
                  '#8e44ad', '#34495e', '#5dade2'] # 8 colors for Sporadic

# NF2 'orig.ident's'
nf2_orig_ids = ['scVS01', 'scVS02', 'snVS01', 'snVS02', 'snVS03',
                'snVS04', 'snVS05', 'snVS06', 'snVS07', 'snVS08', 'snVS09']

# Sporadic 'orig.ident's'
sporadic_orig_ids = ['scVS10', 'scVS11', 'snVS10', 'snVS11', 'snVS12',
                    'snVS13', 'snVS14', 'snVS15']

# Create color dictionary
color_dict = {}

for orig_id, color in zip(nf2_orig_ids, nf2_colors):
    color_dict[orig_id] = color

for orig_id, color in zip(sporadic_orig_ids, sporadic_colors):
    color_dict[orig_id] = color

# Print the color dictionary
print(color_dict)

```

```

{'scVS01': '#e74c3c', 'scVS02': '#d35400', 'snVS01': '#c0392b', 'snVS02':
'#e67e22', 'snVS03': '#f39c12', 'snVS04': '#cd6155', 'snVS05': '#a93226',
'snVS06': '#ba4a00', 'snVS07': '#f1c40f', 'snVS08': '#d68910', 'snVS09':
'#b9770e', 'scVS10': '#3498db', 'scVS11': '#2980b9', 'snVS10': '#1abc9c',
'snVS11': '#2ecc71', 'snVS12': '#16a085', 'snVS13': '#8e44ad', 'snVS14':
'#34495e', 'snVS15': '#5dade2'}

```

```

[45]: ## Kim vs our Schwann subclasses
fig, (ax1, ax2) = plt.subplots(nrows=1,ncols=2,figsize=(10,5), dpi=600, )
ax1_dict = sc.pl.umap(schwann,
                      use_raw = False,
                      frameon=False,
                      show=False,
                      ax = ax1,
                      color = "schwann_pred.type",
                      s = 25,
                      palette = schwann_dict1,
                      edgecolor = 'black',
                      linewidth= 0.5,
                      alpha = 0.9,
                      )
ax2_dict = sc.pl.umap(schwann,
                      use_raw = False,
                      frameon=False,
                      show=False,
                      ax = ax2,
                      color = "schwann_subclasses",
                      s = 25,
                      palette = schwann_dict,
                      edgecolor = 'black',
                      linewidth= 0.5,
                      alpha = 0.9,
                      )

handles, labels = plt.gca().get_legend_handles_labels()

leg1 = ax1.legend(prop = {'size':'14'}, markerscale=1.6, frameon=False,
↳bbox_to_anchor=(0, 1))
for lh in leg1.legend_handles:
    lh.set_alpha(0.9)
    lh.set_edgecolor("black")

leg2 = ax2.legend(prop = {'size':'14'}, markerscale=1.6, frameon=False,
↳bbox_to_anchor=(1, 1))
for lh in leg2.legend_handles:
    lh.set_alpha(0.9)
    lh.set_edgecolor("black")

ax1.set_title("Kim", fontweight="bold", fontsize=24)
ax2.set_title("Our", fontweight="bold", fontsize=24)

#fig.savefig('figures/figure 3/Schwann_S_G2M_score.pdf',bbox_inches='tight')
#fig.savefig('figures/figure 3/Schwann_S_G2M_score.png',bbox_inches='tight')

```

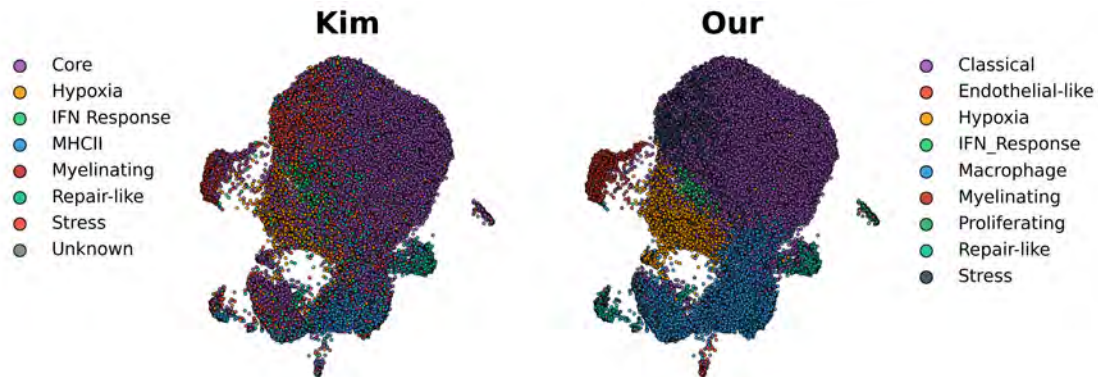
/Users/rampallii2/anaconda3/lib/python3.11/site-

```
packages/scanpy/plotting/_tools/scatterplots.py:394: UserWarning: No data for
colormapping provided via 'c'. Parameters 'cmap' will be ignored
```

```
cax = scatter(
/Users/rampallii2/anaconda3/lib/python3.11/site-
packages/scanpy/plotting/_tools/scatterplots.py:394: UserWarning: No data for
colormapping provided via 'c'. Parameters 'cmap' will be ignored
```

```
cax = scatter(
```

```
[45]: Text(0.5, 1.0, 'Our')
```



```
[ ]: ## Kim vs our Macrophage subclasses
fig, (ax1, ax2) = plt.subplots(nrows=1,ncols=2,figsize=(10,5), dpi=600, )
ax1_dict = sc.pl.umap(macro,
                      use_raw = False,
                      frameon=False,
                      show=False,
                      ax = ax1,
                      color = "Kim_myeloid",
                      s = 25,
                      #palette = macro_dict,
                      edgecolor = 'black',
                      linewidth= 0.5,
                      alpha = 0.9,
                      )
ax2_dict = sc.pl.umap(macro,
                      use_raw = False,
                      frameon=False,
                      show=False,
                      ax = ax2,
                      color = "macrophage_subclasses",
                      s = 25,
                      palette = macro_dict,
                      edgecolor = 'black',
```

```

        linewidth= 0.5,
        alpha = 0.9,
    )

handles, labels = plt.gca().get_legend_handles_labels()

leg1 = ax1.legend(prop = {'size':'14'}, markerscale=1.6, frameon=False,
    ↳bbox_to_anchor=(0, 1))
for lh in leg1.legend_handles:
    lh.set_alpha(0.9)
    lh.set_edgecolor("black")

leg2 = ax2.legend(prop = {'size':'14'}, markerscale=1.6, frameon=False,
    ↳bbox_to_anchor=(1, 1))
for lh in leg2.legend_handles:
    lh.set_alpha(0.9)
    lh.set_edgecolor("black")

ax1.set_title("Kim", fontweight="bold", fontsize=24)
ax2.set_title("Our", fontweight="bold", fontsize=24)

#fig.savefig('figures/figure 3/Schwann_S_G2M_score.pdf',bbox_inches='tight')
#fig.savefig('figures/figure 3/Schwann_S_G2M_score.png',bbox_inches='tight')

```

```

[10]: snVS03_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS03_Schwann_GEX.h5ad")
snVS01_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS01_Schwann_GEX.h5ad")
snVS11_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS11_Schwann_GEX.h5ad")
snVS10_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS10_Schwann_GEX.h5ad")
snVS09_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS09_Schwann_GEX.h5ad")
snVS04_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS04_Schwann_GEX.h5ad")
snVS05_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS05_Schwann_GEX.h5ad")
snVS14_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS14_Schwann_GEX.h5ad")
snVS15_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS15_Schwann_GEX.h5ad")
snVS12_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS12_Schwann_GEX.h5ad")
snVS13_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
    ↳snVS13_Schwann_GEX.h5ad")

```

```

snVS06_schwann = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS06_Schwann_GEX.h5ad")

snVS03 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS03_GEX.h5ad")
snVS01 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS01_GEX.h5ad")
snVS11 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS11_GEX.h5ad")
snVS10 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS10_GEX.h5ad")
snVS09 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS09_GEX.h5ad")
snVS04 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS04_GEX.h5ad")
snVS05 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS05_GEX.h5ad")
snVS14 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS14_GEX.h5ad")
snVS15 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS15_GEX.h5ad")
snVS12 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS12_GEX.h5ad")
snVS13 = sc.read_h5ad("data/Ihika_Analysis/results/012524_h5seurat_h5ad/
↳snVS13_GEX.h5ad")

```

```

/Users/rampallii2/anaconda3/lib/python3.11/site-
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from
.uns['neighbors']['distances'] to .obs['distances'].

```

This is where adjacency matrices should go now.

```

warn(
/Users/rampallii2/anaconda3/lib/python3.11/site-
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from
.uns['neighbors']['distances'] to .obs['distances'].

```

This is where adjacency matrices should go now.

```

warn(
/Users/rampallii2/anaconda3/lib/python3.11/site-
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from
.uns['neighbors']['distances'] to .obs['distances'].

```

This is where adjacency matrices should go now.

```

warn(
/Users/rampallii2/anaconda3/lib/python3.11/site-
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from
.uns['neighbors']['distances'] to .obs['distances'].

```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

```
This is where adjacency matrices should go now.  
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

This is where adjacency matrices should go now.

```
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

This is where adjacency matrices should go now.

```
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

This is where adjacency matrices should go now.

```
warn(  
/Users/rampallii2/anaconda3/lib/python3.11/site-  
packages/anndata/compat/__init__.py:298: FutureWarning: Moving element from  
.uns['neighbors']['distances'] to .obsp['distances'].
```

This is where adjacency matrices should go now.

```
warn(  

```

```
[12]: ## Plot NF2 depletion score on individual samples  
fig, ((ax1,ax2,ax3), (ax4,ax5,ax6)) = plt.  
↳subplots(nrows=2,ncols=3,figsize=(5,3), dpi=600)  
ax1_dict = sc.pl.umap(snVS03,  
                        use_raw = False,  
                        frameon=False,  
                        show=False,  
                        ax = ax1,  
                        color = "X22q",  
                        s = 3,  
                        cmap='viridis',  
                        #edgecolor = 'black',  
                        #linewidth= 0.5,  
                        alpha = 0.9,  
                        colorbar_loc=None,  
                        # vmin=0,  
                        # vmax=0.5  
                        )  
  
ax2_dict = sc.pl.umap(snVS05,  
                        use_raw = False,  
                        frameon=False,  
                        show=False,  
                        ax = ax2,  
                        color = "X22q",
```

```

        s = 3,
        cmap='viridis',
        #edgecolor = 'black',
        #linewidth= 0.5,
        alpha = 0.9,
        colorbar_loc=None,
        #vmin=0,
        #vmax=0.5
    )
ax3_dict = sc.pl.umap(snVS01,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax3,
    color = "X22q",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    #vmin=0,
    #vmax=0.5
)
ax4_dict = sc.pl.umap(snVS13,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax4,
    color = "X22q",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    #vmin=0,
    #vmax=0.5
)
ax5_dict = sc.pl.umap(snVS10,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax5,
    color = "X22q",
    s = 3,
    cmap='viridis',

```

```

        # edgecolor = 'black',
        #linewidth= 0.5,
        alpha = 0.9,
        colorbar_loc=None,
        # vmin=0,
        #vmax=0.5
    )

ax6_dict = sc.pl.umap(snVS14,
                      use_raw = False,
                      frameon=False,
                      show=False,
                      ax = ax6,
                      color = "X22q",
                      s = 3,
                      cmap='viridis',
                      #edgecolor = 'black',
                      #linewidth= 0.5,
                      colorbar_loc=None,
                      alpha = 0.9,
                      # vmin=0,
                      #vmax=0.5
    )

ax1.set_title("")
ax2.set_title("")
ax3.set_title("")
ax4.set_title("")
ax5.set_title("")
ax6.set_title("")
fig.savefig('figures/figure 3/X22qConicsmat_030501131014.
↳pdf',bbox_inches='tight')
fig.savefig('figures/figure 3/X22qConicsmat_030501131014.
↳pdf',bbox_inches='tight')

```



```
[13]: ## Plot NF2 depletion score on individual samples
fig, ((ax1,ax2,ax3), (ax4,ax5,ax6)) = plt.
    subplots(nrows=2,ncols=3,figsize=(5,3), dpi=600)
ax1_dict = sc.pl.umap(snVS03,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax1,
    color = "score21",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    # vmin=0,
    # vmax=0.5
)

ax2_dict = sc.pl.umap(snVS05,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax2,
    color = "score21",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
```

```

        colorbar_loc=None,
        #vmin=0,
        #vmax=0.5
    )
ax3_dict = sc.pl.umap(snVS01,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax3,
    color = "score21",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    # vmin=0,
    #vmax=0.5
)
ax4_dict = sc.pl.umap(snVS13,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax4,
    color = "score21",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    #vmin=0,
    #vmax=0.5
)
ax5_dict = sc.pl.umap(snVS10,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax5,
    color = "score21",
    s = 3,
    cmap='viridis',
    # edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    # vmin=0,

```

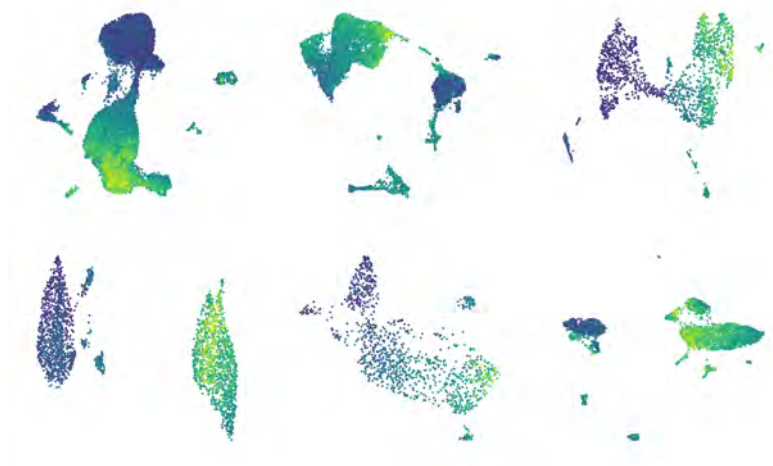
```

        #vmax=0.5
    )

ax6_dict = sc.pl.umap(snVS14,
                      use_raw = False,
                      frameon=False,
                      show=False,
                      ax = ax6,
                      color = "score21",
                      s = 3,
                      cmap='viridis',
                      #edgecolor = 'black',
                      #linewidth= 0.5,
                      colorbar_loc=None,
                      alpha = 0.9,
                      # vmin=0,
                      #vmax=0.5
    )

ax1.set_title("")
ax2.set_title("")
ax3.set_title("")
ax4.set_title("")
ax5.set_title("")
ax6.set_title("")
fig.savefig('figures/figure 3/NF2score_030501131014.pdf',bbox_inches='tight')
fig.savefig('figures/figure 3/NF2score_030501131014.png',bbox_inches='tight')

```



[14]: *## Plot NF2 depletion score on individual samples with validation*

```

fig, ((ax1,ax2,ax3,ax4,ax5)) = plt.subplots(nrows=1,ncols=5,figsize=(15,3),
    ↪dpi=600)
ax1_dict = sc.pl.umap(snVS03,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax1,
    color = "score21",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    vmin=0,
    # vmax=0.5
)

ax2_dict = sc.pl.umap(snVS03,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax2,
    color = "HS011",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    vmin=0,
    #vmax=0.5
)

ax3_dict = sc.pl.umap(snVS03,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax3,
    color = "HS111",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    vmin=0,
    #vmax=0.5
)

```

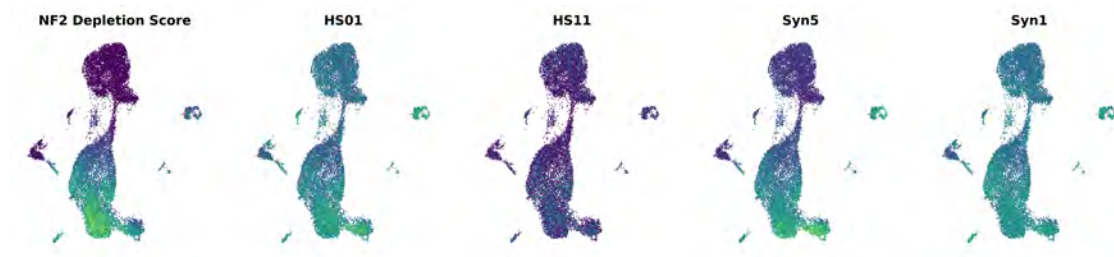
```

    )
ax4_dict = sc.pl.umap(snVS03,
                      use_raw = False,
                      frameon=False,
                      show=False,
                      ax = ax4,
                      color = "Syn51",
                      s = 3,
                      cmap='viridis',
                      #edgecolor = 'black',
                      #linewidth= 0.5,
                      alpha = 0.9,
                      colorbar_loc=None,
                      vmin=0,
                      #vmax=0.5
                      )
ax5_dict = sc.pl.umap(snVS03,
                      use_raw = False,
                      frameon=False,
                      show=False,
                      ax = ax5,
                      color = "Syn11",
                      s = 3,
                      cmap='viridis',
                      # edgecolor = 'black',
                      #linewidth= 0.5,
                      alpha = 0.9,
                      colorbar_loc=None,
                      vmin=0,
                      #vmax=0.5
                      )

ax1.set_title("NF2 Depletion Score", fontweight="bold", fontsize=10)
ax2.set_title("HS01", fontweight="bold", fontsize=10)
ax3.set_title("HS11", fontweight="bold", fontsize=10)
ax4.set_title("Syn5", fontweight="bold", fontsize=10)
ax5.set_title("Syn1", fontweight="bold", fontsize=10)
#ax6.set_title("")
#fig.savefig('figures/figure 3/NF2score_030501131014.pdf',bbox_inches='tight')
#fig.savefig('figures/figure 3/NF2score_snVS03_validation.
    ↪pdf',bbox_inches='tight')

```

[14]: Text(0.5, 1.0, 'Syn1')



```
[17]: ## Plot Hippo depletion score on individual samples
fig, ((ax1,ax2,ax3), (ax4,ax5,ax6)) = plt.
    subplots(nrows=2,ncols=3,figsize=(5,3), dpi=600)
ax1_dict = sc.pl.umap(snVS03,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax1,
    color = "Hippo1",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    # vmin=0,
    # vmax=0.5
)

ax2_dict = sc.pl.umap(snVS05,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax2,
    color = "Hippo1",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    #vmin=0,
    #vmax=0.5
)

ax3_dict = sc.pl.umap(snVS01,
    use_raw = False,
```

```

        frameon=False,
        show=False,
        ax = ax3,
        color = "Hippo1",
        s = 3,
        cmap='viridis',
        #edgecolor = 'black',
        #linewidth= 0.5,
        alpha = 0.9,
        colorbar_loc=None,
        # vmin=0,
        #vmax=0.5
    )
ax4_dict = sc.pl.umap(snVS13,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax4,
    color = "Hippo1",
    s = 3,
    cmap='viridis',
    #edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    #vmin=0,
    #vmax=0.5
)
ax5_dict = sc.pl.umap(snVS10,
    use_raw = False,
    frameon=False,
    show=False,
    ax = ax5,
    color = "Hippo1",
    s = 3,
    cmap='viridis',
    # edgecolor = 'black',
    #linewidth= 0.5,
    alpha = 0.9,
    colorbar_loc=None,
    # vmin=0,
    #vmax=0.5
)
ax6_dict = sc.pl.umap(snVS14,
    use_raw = False,
    frameon=False,

```

```

        show=False,
        ax = ax6,
        color = "Hippo1",
        s = 3,
        cmap='viridis',
        #edgecolor = 'black',
        #linewidth= 0.5,
        colorbar_loc=None,
        alpha = 0.9,
        # vmin=0,
        #vmax=0.5
    )

ax1.set_title("")
ax2.set_title("")
ax3.set_title("")
ax4.set_title("")
ax5.set_title("")
ax6.set_title("")

```

```
[17]: Text(0.5, 1.0, '')
```



0.1 Define MDS Subclasses

```

[16]: # create new column "NF2_status". Cells below cutoff (no NF2 depletion) will be
      ↪ annotated as 0
def NF2_status(row):
    if row['new_mds'] >= 0.642159:
        return "NF2_depleted"
    elif row['score21'] <= 0.169553:

```

```

        return "NF2_intact"
    else:
        # No need to check the condition again; if it reaches here, it means
        ↪ it's neither depleted nor intact
        return "NF2_intermediate"

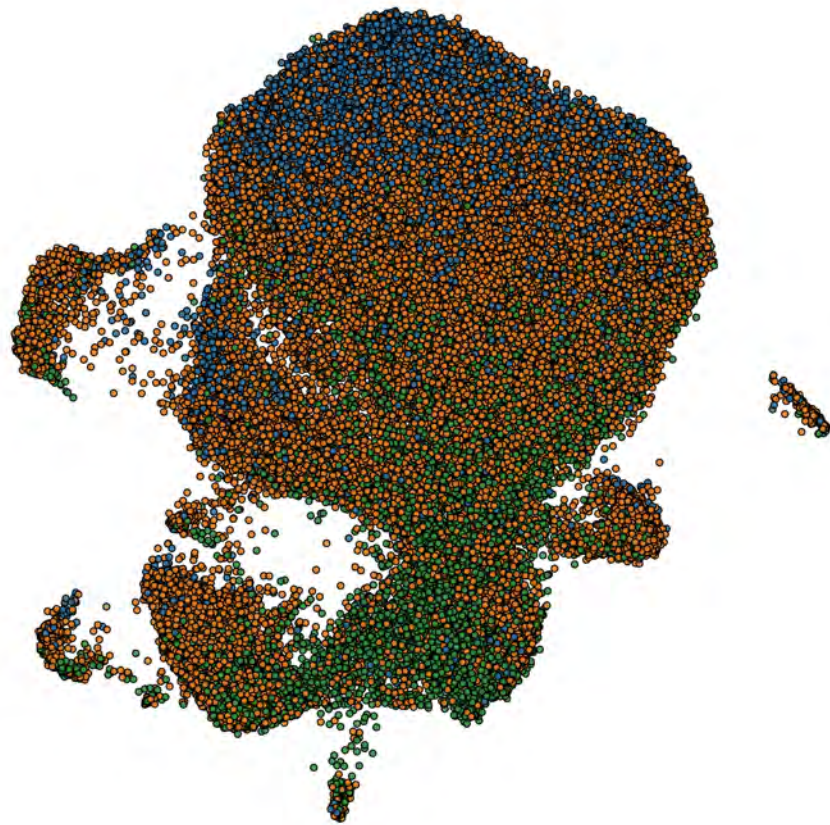
# Assuming 'schwann.obs' is a DataFrame
schwann.obs['NF2_status_new'] = schwann.obs.apply(NF2_status, axis=1)

```

```

[28]: mds_dict = {'NF2_depleted': '#3274a2',
                 'NF2_intermediate': '#e1812b',
                 'NF2_intact': '#3a9144'}
fig, ax1 = plt.subplots(nrows=1,ncols=1,figsize=(6,6), dpi=600)
ax1_dict = sc.pl.umap(schwann,
                      use_raw = False,
                      frameon=False,
                      show=False,
                      ax = ax1,
                      color = "NF2_status_new",
                      s = 25,
                      cmap='viridis',
                      edgecolor = 'black',
                      linewidth= 0.5,
                      alpha = 0.9,
                      palette=mds_dict,
                      legend_loc=None
                      )
ax1.set_title('')
fig.savefig('new_mds_status.png', bbox_inches='tight')

```



```
[10]: import statsmodels.api as sm
x=schwann.obs['score21']
y=schwann.obs['new_mds']
fig, ax = plt.subplots(1,1, figsize=(4,4))
model = sm.OLS(y, sm.add_constant(x))
results = model.fit()

print(results.params)
print(results.summary())
b, m = results.params

correlation = x.corr(y)
print(f'Correlation={correlation}')

sns.scatterplot(x=x, y=y, color='black', ax=ax)
xline = np.linspace(x.min(), x.max(), 100)
ax.plot(xline, m * xline + b, color='black',
        label=f'$y = {m:.2f}x + {b:.2f}$')
```

```
#ax.axline(xy1=(0, b), slope=m, label=f'$y = {m:.1f}x {b:+.1f}$')
ax.legend(fontsize=6)
ax.set_ylabel('Parsimonious MDS')
ax.set_xlabel('Current MDS')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
fig.savefig('compare_mds.png', bbox_inches='tight')
```

```
const      0.032262
score21    1.305135
dtype: float64
```

OLS Regression Results

```
=====
Dep. Variable:          new_mds    R-squared:                0.792
Model:                  OLS        Adj. R-squared:            0.792
Method:                 Least Squares   F-statistic:              1.895e+05
Date:                  Wed, 08 Jul 2026   Prob (F-statistic):       0.00
Time:                  21:40:02         Log-Likelihood:           40201.
No. Observations:      49639          AIC:                     -8.040e+04
Df Residuals:          49637          BIC:                     -8.038e+04
Df Model:               1
Covariance Type:       nonrobust
=====
```

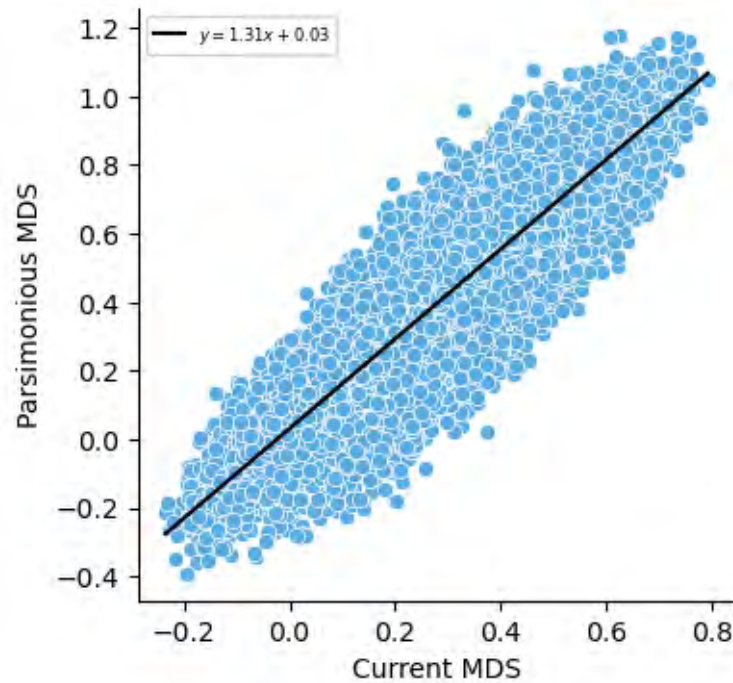
	coef	std err	t	P> t	[0.025	0.975]
const	0.0323	0.001	32.757	0.000	0.030	0.034
score21	1.3051	0.003	435.321	0.000	1.299	1.311

```
=====
Omnibus:                43.873    Durbin-Watson:           1.939
Prob(Omnibus):          0.000    Jarque-Bera (JB):        47.892
Skew:                   -0.038    Prob(JB):                3.98e-11
Kurtosis:               3.131    Cond. No.:               6.73
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Correlation=0.8901892286400743



```
[7]: schwann
```

```
[7]: AnnData object with n_obs × n_vars = 49639 × 38224
      obs: 'nCount_RNA', 'nFeature_RNA', 'orig.ident', 'type', 'percent.mt',
      'pANN_0.25_0.09_0', 'DF.classifications_0.25_0.09_0', 'RNA_snn_res.0.5',
      'seurat_clusters', 'pred.type', 'score1', 'X1p', 'X1q', 'X2p', 'X2q', 'X3p',
      'X3q', 'X4p', 'X4q', 'X5p', 'X5q', 'X6p', 'X6q', 'X7p', 'X7q', 'X8p', 'X8q',
      'X9p', 'X9q', 'X10p', 'X10q', 'X11p', 'X11q', 'X12p', 'X12q', 'X13q', 'X14q',
      'X15q', 'X16p', 'X16q', 'X17p', 'X17q', 'X18q', 'X19p', 'X19q', 'X20p', 'X20q',
      'X21q', 'X22q', 'x22q', 'integrated_snn_res.0.5', 'integrated_snn_res.1',
      'schwann_pred.type', 'integrated_snn_res.0.3', 'S.Score', 'G2M.Score', 'Phase',
      'old.ident', 'integrated_snn_res.0.1', 'schwann_subclasses', 'score21',
      'Hippo1', 'score', 'new_mds'
      var: 'features'
      obsm: 'X_umap'
```

```
[13]: import statsmodels.api as sm
      x=schwann.obs['Hippo1']
      y=schwann.obs['score21']
      fig, ax = plt.subplots(1,1, figsize=(4,4))
      model = sm.OLS(y, sm.add_constant(x))
      results = model.fit()

      print(results.params)
```

```

print(results.summary())
b, m = results.params

correlation = x.corr(y)
print(f'Correlation={correlation}')

sns.scatterplot(x=x, y=y, color=color, ax=ax)
xline = np.linspace(x.min(), x.max(), 100)
ax.plot(xline, m * xline + b, color='black',
        label=f'$y = {m:.2f}x {b:+.2f}$')
#ax.axline(xy1=(0, b), slope=m, label=f'$y = {m:.1f}x {b:+.1f}$')
ax.legend(fontsize=6)
ax.set_ylabel('Hippo')
ax.set_xlabel('MDS')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
fig.savefig('compare_mds_hippo.png', bbox_inches='tight')

```

```

const      0.243169
Hippo1     0.315192
dtype: float64

```

OLS Regression Results

```

=====
Dep. Variable:          score21    R-squared:                0.086
Model:                  OLS        Adj. R-squared:            0.086
Method:                 Least Squares    F-statistic:            4678.
Date:                  Wed, 08 Jul 2026    Prob (F-statistic):      0.00
Time:                  22:04:04          Log-Likelihood:          22406.
No. Observations:      49639           AIC:                    -4.481e+04
Df Residuals:          49637           BIC:                    -4.479e+04
Df Model:               1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.2432	0.001	259.963	0.000	0.241	0.245
Hippo1	0.3152	0.005	68.398	0.000	0.306	0.324

```

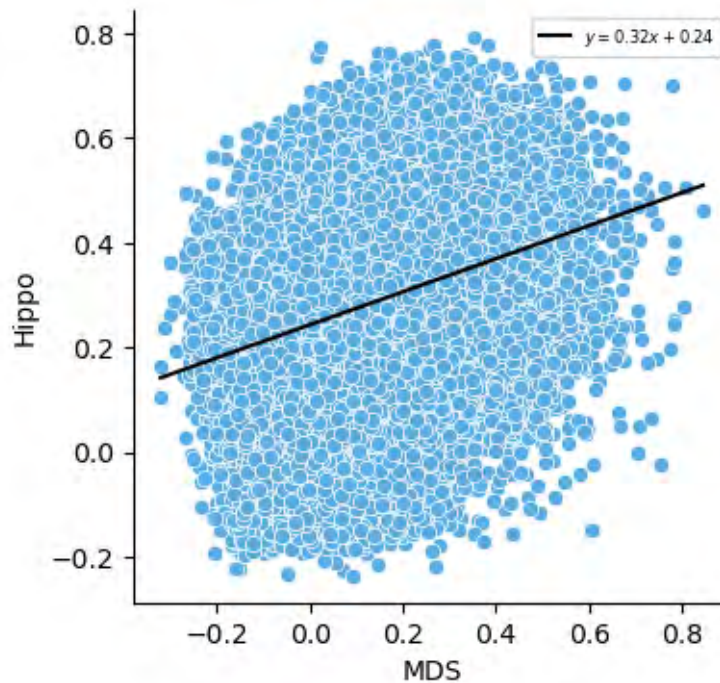
=====
Omnibus:                 336.733    Durbin-Watson:           1.372
Prob(Omnibus):            0.000    Jarque-Bera (JB):        336.036
Skew:                     -0.190    Prob(JB):                 1.07e-73
Kurtosis:                 2.863     Cond. No.                  6.79
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Correlation=0.2934806875042572



```
[15]: import statsmodels.api as sm
x=schwann.obs['Hippo1']
y=schwann.obs['new_mds']
fig, ax = plt.subplots(1,1, figsize=(4,4))
model = sm.OLS(y, sm.add_constant(x))
results = model.fit()

print(results.params)
print(results.summary())
b, m = results.params

correlation = x.corr(y)
print(f'Correlation={correlation}')

sns.scatterplot(x=x, y=y, color='blue', ax=ax)
xline = np.linspace(x.min(), x.max(), 100)
ax.plot(xline, m * xline + b, color='black',
        label=f'$y = {m:.2f}x + {b:.2f}$')
#ax.axline(xy1=(0, b), slope=m, label=f'$y = {m:.1f}x + {b:.1f}$')
ax.legend(fontsize=6)
ax.set_ylabel('Hippo')
ax.set_xlabel('Parismonious MDS')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
```

```
fig.savefig('compare_parsimonious_mds_hippo.png', bbox_inches='tight')
```

```
const      0.348623
Hippo1     0.418741
dtype: float64
```

OLS Regression Results

```
=====
Dep. Variable:          new_mds    R-squared:                0.071
Model:                  OLS        Adj. R-squared:             0.071
Method:                 Least Squares    F-statistic:            3778.
Date:                   Wed, 08 Jul 2026    Prob (F-statistic):      0.00
Time:                   22:07:38          Log-Likelihood:         2997.5
No. Observations:      49639          AIC:                   -5991.
Df Residuals:          49637          BIC:                   -5973.
Df Model:               1
Covariance Type:        nonrobust
=====
```

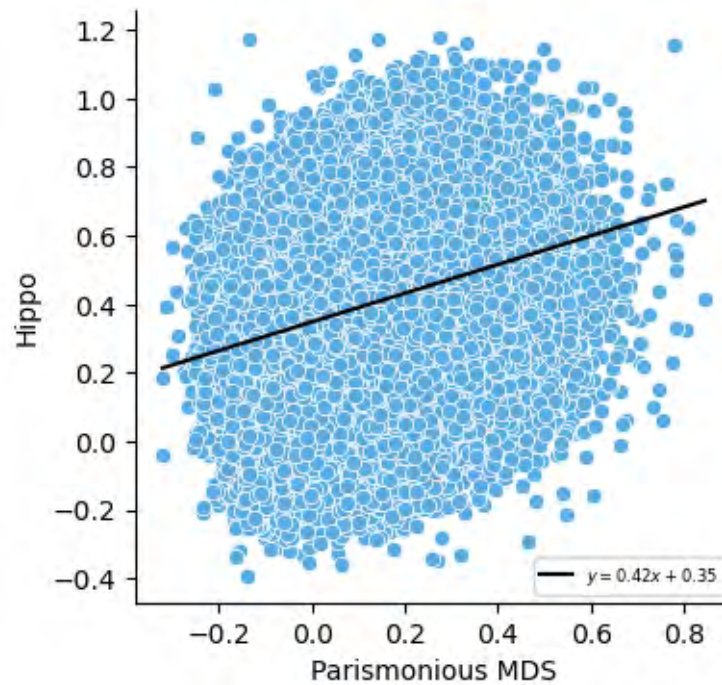
	coef	std err	t	P> t	[0.025	0.975]
const	0.3486	0.001	252.089	0.000	0.346	0.351
Hippo1	0.4187	0.007	61.462	0.000	0.405	0.432

```
=====
Omnibus:                264.861    Durbin-Watson:           1.558
Prob(Omnibus):           0.000    Jarque-Bera (JB):        227.002
Skew:                    -0.111    Prob(JB):                5.10e-50
Kurtosis:                2.754    Cond. No.                6.79
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Correlation=0.2659356565366325



```
[24]: import statsmodels.api as sm
import numpy as np

x = schwann.obs['score21'].values

# Extract TEAD1 expression; handle sparse .X
gene_idx = schwann.var_names.get_loc('TEAD1')
y = schwann.X[:, gene_idx]
y = np.asarray(y.todense()).ravel() if hasattr(y, 'todense') else np.asarray(y).
    ↪ravel()

# Drop NaNs for a clean fit
mask = np.isfinite(x) & np.isfinite(y)
x, y = x[mask], y[mask]

color = 'steelblue'

fig, ax = plt.subplots(1, 1, figsize=(4, 4))
model = sm.OLS(y, sm.add_constant(x))
results = model.fit()
print(results.params)
print(results.summary())

b, m = results.params
```

```

correlation = np.corrcoef(x, y)[0, 1]
print(f'Correlation={correlation}')

sns.scatterplot(x=x, y=y, color=color, ax=ax, s=10, alpha=0.5)
xline = np.linspace(x.min(), x.max(), 100)
ax.plot(xline, m * xline + b, color='black', label=f'$y = {m:.2f}x + {b:.2f}$')
ax.legend(fontsize=6)
ax.set_ylabel('TEAD1 expression')
ax.set_xlabel('MDS')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
fig.savefig('compare_mds_tead1.png', dpi=300, bbox_inches='tight')

```

[1.50901939 1.8868281]

OLS Regression Results

=====						
Dep. Variable:	y	R-squared:	0.070			
Model:	OLS	Adj. R-squared:	0.070			
Method:	Least Squares	F-statistic:	3746.			
Date:	Thu, 09 Jul 2026	Prob (F-statistic):	0.00			
Time:	09:15:11	Log-Likelihood:	-75481.			
No. Observations:	49639	AIC:	1.510e+05			
Df Residuals:	49637	BIC:	1.510e+05			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	1.5090	0.010	149.002	0.000	1.489	1.529
x1	1.8868	0.031	61.203	0.000	1.826	1.947
=====						
Omnibus:	3616.356	Durbin-Watson:	1.905			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	3168.881			
Skew:	-0.547	Prob(JB):	0.00			
Kurtosis:	2.422	Cond. No.	6.73			
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Correlation=0.26489351410105727

